```python
# Copyright (c) OpenMMLab. All rights reserved.
import torch
import torch.nn as nn
from mmcv.cnn import (ConvModule, DepthwiseSeparableConvModule,
                      build_activation_layer, build_norm_layer)
from mmengine.model import BaseModule

from mmseg.registry import MODELS
from ..utils import resize


class DetailBranch(BaseModule):
    """Detail Branch with wide channels and shallow layers to capture low-level
    details and generate high-resolution feature representation.

    Args:
        detail_channels (Tuple[int]): Size of channel numbers of each stage
            in Detail Branch, in paper it has 3 stages.
            Default: (64, 64, 128).
        in_channels (int): Number of channels of input image. Default: 3.
        conv_cfg (dict | None): Config of conv layers.
            Default: None.
        norm_cfg (dict | None): Config of norm layers.
            Default: dict(type='BN').
        act_cfg (dict): Config of activation layers.
            Default: dict(type='ReLU').
        init_cfg (dict or list[dict], optional): Initialization config dict.
            Default: None.
    Returns:
        x (torch.Tensor): Feature map of Detail Branch.
    """

    def __init__(self,
                 detail_channels=(64, 64, 128),
                 in_channels=3,
                 conv_cfg=None,
                 norm_cfg=dict(type='BN'),
                 act_cfg=dict(type='ReLU'),
                 init_cfg=None):
        super().__init__(init_cfg=init_cfg)
        detail_branch = []
        for i in range(len(detail_channels)):
            if i == 0:
                detail_branch.append(
                    nn.Sequential(
                        ConvModule(
                            in_channels=in_channels,
                            out_channels=detail_channels[i],
                            kernel_size=3,
                            stride=2,
                            padding=1,
                            conv_cfg=conv_cfg,
                            norm_cfg=norm_cfg,
                            act_cfg=act_cfg),
                        ConvModule(
                            in_channels=detail_channels[i],
                            out_channels=detail_channels[i],
                            kernel_size=3,
                            stride=1,
                            padding=1,
                            conv_cfg=conv_cfg,
                            norm_cfg=norm_cfg,
                            act_cfg=act_cfg)))
            else:
                detail_branch.append(
                    nn.Sequential(
                        ConvModule(
                            in_channels=detail_channels[i - 1],
                            out_channels=detail_channels[i],
```

```
                                kernel_size=3,
                                stride=2,
                                padding=1,
                                conv_cfg=conv_cfg,
                                norm_cfg=norm_cfg,
                                act_cfg=act_cfg),
                            ConvModule(
                                in_channels=detail_channels[i],
                                out_channels=detail_channels[i],
                                kernel_size=3,
                                stride=1,
                                padding=1,
                                conv_cfg=conv_cfg,
                                norm_cfg=norm_cfg,
                                act_cfg=act_cfg),
                            ConvModule(
                                in_channels=detail_channels[i],
                                out_channels=detail_channels[i],
                                kernel_size=3,
                                stride=1,
                                padding=1,
                                conv_cfg=conv_cfg,
                                norm_cfg=norm_cfg,
                                act_cfg=act_cfg)))
        self.detail_branch = nn.ModuleList(detail_branch)

    def forward(self, x):
        for stage in self.detail_branch:
            x = stage(x)
        return x


class StemBlock(BaseModule):
    """Stem Block at the beginning of Semantic Branch.

    Args:
        in_channels (int): Number of input channels.
            Default: 3.
        out_channels (int): Number of output channels.
            Default: 16.
        conv_cfg (dict | None): Config of conv layers.
            Default: None.
        norm_cfg (dict | None): Config of norm layers.
            Default: dict(type='BN').
        act_cfg (dict): Config of activation layers.
            Default: dict(type='ReLU').
        init_cfg (dict or list[dict], optional): Initialization config dict.
            Default: None.
    Returns:
        x (torch.Tensor): First feature map in Semantic Branch.
    """

    def __init__(self,
                 in_channels=3,
                 out_channels=16,
                 conv_cfg=None,
                 norm_cfg=dict(type='BN'),
                 act_cfg=dict(type='ReLU'),
                 init_cfg=None):
        super().__init__(init_cfg=init_cfg)

        self.conv_first = ConvModule(
            in_channels=in_channels,
            out_channels=out_channels,
            kernel_size=3,
            stride=2,
            padding=1,
            conv_cfg=conv_cfg,
            norm_cfg=norm_cfg,
```

```
            act_cfg=act_cfg)
        self.convs = nn.Sequential(
            ConvModule(
                in_channels=out_channels,
                out_channels=out_channels // 2,
                kernel_size=1,
                stride=1,
                padding=0,
                conv_cfg=conv_cfg,
                norm_cfg=norm_cfg,
                act_cfg=act_cfg),
            ConvModule(
                in_channels=out_channels // 2,
                out_channels=out_channels,
                kernel_size=3,
                stride=2,
                padding=1,
                conv_cfg=conv_cfg,
                norm_cfg=norm_cfg,
                act_cfg=act_cfg))
        self.pool = nn.MaxPool2d(
            kernel_size=3, stride=2, padding=1, ceil_mode=False)
        self.fuse_last = ConvModule(
            in_channels=out_channels * 2,
            out_channels=out_channels,
            kernel_size=3,
            stride=1,
            padding=1,
            conv_cfg=conv_cfg,
            norm_cfg=norm_cfg,
            act_cfg=act_cfg)

    def forward(self, x):
        x = self.conv_first(x)
        x_left = self.convs(x)
        x_right = self.pool(x)
        x = self.fuse_last(torch.cat([x_left, x_right], dim=1))
        return x


class GELayer(BaseModule):
    """Gather-and-Expansion Layer.

    Args:
        in_channels (int): Number of input channels.
        out_channels (int): Number of output channels.
        exp_ratio (int): Expansion ratio for middle channels.
            Default: 6.
        stride (int): Stride of GELayer. Default: 1
        conv_cfg (dict | None): Config of conv layers.
            Default: None.
        norm_cfg (dict | None): Config of norm layers.
            Default: dict(type='BN').
        act_cfg (dict): Config of activation layers.
            Default: dict(type='ReLU').
        init_cfg (dict or list[dict], optional): Initialization config dict.
            Default: None.
    Returns:
        x (torch.Tensor): Intermediate feature map in
            Semantic Branch.
    """

    def __init__(self,
                 in_channels,
                 out_channels,
                 exp_ratio=6,
                 stride=1,
                 conv_cfg=None,
                 norm_cfg=dict(type='BN'),
```

```
                 act_cfg=dict(type='ReLU'),
                 init_cfg=None):
        super().__init__(init_cfg=init_cfg)
        mid_channel = in_channels * exp_ratio
        self.conv1 = ConvModule(
            in_channels=in_channels,
            out_channels=in_channels,
            kernel_size=3,
            stride=1,
            padding=1,
            conv_cfg=conv_cfg,
            norm_cfg=norm_cfg,
            act_cfg=act_cfg)
        if stride == 1:
            self.dwconv = nn.Sequential(
                # ReLU in ConvModule not shown in paper
                ConvModule(
                    in_channels=in_channels,
                    out_channels=mid_channel,
                    kernel_size=3,
                    stride=stride,
                    padding=1,
                    groups=in_channels,
                    conv_cfg=conv_cfg,
                    norm_cfg=norm_cfg,
                    act_cfg=act_cfg))
            self.shortcut = None
        else:
            self.dwconv = nn.Sequential(
                ConvModule(
                    in_channels=in_channels,
                    out_channels=mid_channel,
                    kernel_size=3,
                    stride=stride,
                    padding=1,
                    groups=in_channels,
                    bias=False,
                    conv_cfg=conv_cfg,
                    norm_cfg=norm_cfg,
                    act_cfg=None),
                # ReLU in ConvModule not shown in paper
                ConvModule(
                    in_channels=mid_channel,
                    out_channels=mid_channel,
                    kernel_size=3,
                    stride=1,
                    padding=1,
                    groups=mid_channel,
                    conv_cfg=conv_cfg,
                    norm_cfg=norm_cfg,
                    act_cfg=act_cfg),
            )
            self.shortcut = nn.Sequential(
                DepthwiseSeparableConvModule(
                    in_channels=in_channels,
                    out_channels=out_channels,
                    kernel_size=3,
                    stride=stride,
                    padding=1,
                    dw_norm_cfg=norm_cfg,
                    dw_act_cfg=None,
                    pw_norm_cfg=norm_cfg,
                    pw_act_cfg=None,
                ))

        self.conv2 = nn.Sequential(
            ConvModule(
                in_channels=mid_channel,
                out_channels=out_channels,
```

```
                kernel_size=1,
                stride=1,
                padding=0,
                bias=False,
                conv_cfg=conv_cfg,
                norm_cfg=norm_cfg,
                act_cfg=None,
            ))

        self.act = build_activation_layer(act_cfg)

    def forward(self, x):
        identity = x
        x = self.conv1(x)
        x = self.dwconv(x)
        x = self.conv2(x)
        if self.shortcut is not None:
            shortcut = self.shortcut(identity)
            x = x + shortcut
        else:
            x = x + identity
        x = self.act(x)
        return x


class CEBlock(BaseModule):
    """Context Embedding Block for large receptive filed in Semantic Branch.

    Args:
        in_channels (int): Number of input channels.
            Default: 3.
        out_channels (int): Number of output channels.
            Default: 16.
        conv_cfg (dict | None): Config of conv layers.
            Default: None.
        norm_cfg (dict | None): Config of norm layers.
            Default: dict(type='BN').
        act_cfg (dict): Config of activation layers.
            Default: dict(type='ReLU').
        init_cfg (dict or list[dict], optional): Initialization config dict.
            Default: None.
    Returns:
        x (torch.Tensor): Last feature map in Semantic Branch.
    """

    def __init__(self,
                 in_channels=3,
                 out_channels=16,
                 conv_cfg=None,
                 norm_cfg=dict(type='BN'),
                 act_cfg=dict(type='ReLU'),
                 init_cfg=None):
        super().__init__(init_cfg=init_cfg)
        self.in_channels = in_channels
        self.out_channels = out_channels
        self.gap = nn.Sequential(
            nn.AdaptiveAvgPool2d((1, 1)),
            build_norm_layer(norm_cfg, self.in_channels)[1])
        self.conv_gap = ConvModule(
            in_channels=self.in_channels,
            out_channels=self.out_channels,
            kernel_size=1,
            stride=1,
            padding=0,
            conv_cfg=conv_cfg,
            norm_cfg=norm_cfg,
            act_cfg=act_cfg)
        # Note: in paper here is naive conv2d, no bn-relu
        self.conv_last = ConvModule(
```

```
            in_channels=self.out_channels,
            out_channels=self.out_channels,
            kernel_size=3,
            stride=1,
            padding=1,
            conv_cfg=conv_cfg,
            norm_cfg=norm_cfg,
            act_cfg=act_cfg)

    def forward(self, x):
        identity = x
        x = self.gap(x)
        x = self.conv_gap(x)
        x = identity + x
        x = self.conv_last(x)
        return x


class SemanticBranch(BaseModule):
    """Semantic Branch which is lightweight with narrow channels and deep
    layers to obtain¿high-level semantic context.

    Args:
        semantic_channels(Tuple[int]): Size of channel numbers of
            various stages in Semantic Branch.
            Default: (16, 32, 64, 128).
        in_channels (int): Number of channels of input image. Default: 3.
        exp_ratio (int): Expansion ratio for middle channels.
            Default: 6.
        init_cfg (dict or list[dict], optional): Initialization config dict.
            Default: None.
    Returns:
        semantic_outs (List[torch.Tensor]): List of several feature maps
            for auxiliary heads (Booster) and Bilateral
            Guided Aggregation Layer.
    """

    def __init__(self,
                 semantic_channels=(16, 32, 64, 128),
                 in_channels=3,
                 exp_ratio=6,
                 init_cfg=None):
        super().__init__(init_cfg=init_cfg)
        self.in_channels = in_channels
        self.semantic_channels = semantic_channels
        self.semantic_stages = []
        for i in range(len(semantic_channels)):
            stage_name = f'stage{i + 1}'
            self.semantic_stages.append(stage_name)
            if i == 0:
                self.add_module(
                    stage_name,
                    StemBlock(self.in_channels, semantic_channels[i]))
            elif i == (len(semantic_channels) - 1):
                self.add_module(
                    stage_name,
                    nn.Sequential(
                        GELayer(semantic_channels[i - 1], semantic_channels[i],
                                exp_ratio, 2),
                        GELayer(semantic_channels[i], semantic_channels[i],
                                exp_ratio, 1),
                        GELayer(semantic_channels[i], semantic_channels[i],
                                exp_ratio, 1),
                        GELayer(semantic_channels[i], semantic_channels[i],
                                exp_ratio, 1)))
            else:
                self.add_module(
                    stage_name,
                    nn.Sequential(
```

```
                        GELayer(semantic_channels[i - 1], semantic_channels[i],
                                exp_ratio, 2),
                        GELayer(semantic_channels[i], semantic_channels[i],
                                exp_ratio, 1)))

        self.add_module(f'stage{len(semantic_channels)}_CEBlock',
                        CEBlock(semantic_channels[-1], semantic_channels[-1]))
        self.semantic_stages.append(f'stage{len(semantic_channels)}_CEBlock')

    def forward(self, x):
        semantic_outs = []
        for stage_name in self.semantic_stages:
            semantic_stage = getattr(self, stage_name)
            x = semantic_stage(x)
            semantic_outs.append(x)
        return semantic_outs


class BGALayer(BaseModule):
    """Bilateral Guided Aggregation Layer to fuse the complementary information
    from both Detail Branch and Semantic Branch.

    Args:
        out_channels (int): Number of output channels.
            Default: 128.
        align_corners (bool): align_corners argument of F.interpolate.
            Default: False.
        conv_cfg (dict | None): Config of conv layers.
            Default: None.
        norm_cfg (dict | None): Config of norm layers.
            Default: dict(type='BN').
        act_cfg (dict): Config of activation layers.
            Default: dict(type='ReLU').
        init_cfg (dict or list[dict], optional): Initialization config dict.
            Default: None.
    Returns:
        output (torch.Tensor): Output feature map for Segment heads.
    """

    def __init__(self,
                 out_channels=128,
                 align_corners=False,
                 conv_cfg=None,
                 norm_cfg=dict(type='BN'),
                 act_cfg=dict(type='ReLU'),
                 init_cfg=None):
        super().__init__(init_cfg=init_cfg)
        self.out_channels = out_channels
        self.align_corners = align_corners
        self.detail_dwconv = nn.Sequential(
            DepthwiseSeparableConvModule(
                in_channels=self.out_channels,
                out_channels=self.out_channels,
                kernel_size=3,
                stride=1,
                padding=1,
                dw_norm_cfg=norm_cfg,
                dw_act_cfg=None,
                pw_norm_cfg=None,
                pw_act_cfg=None,
            ))
        self.detail_down = nn.Sequential(
            ConvModule(
                in_channels=self.out_channels,
                out_channels=self.out_channels,
                kernel_size=3,
                stride=2,
                padding=1,
                bias=False,
```

```
                conv_cfg=conv_cfg,
                norm_cfg=norm_cfg,
                act_cfg=None),
            nn.AvgPool2d(kernel_size=3, stride=2, padding=1, ceil_mode=False))
        self.semantic_conv = nn.Sequential(
            ConvModule(
                in_channels=self.out_channels,
                out_channels=self.out_channels,
                kernel_size=3,
                stride=1,
                padding=1,
                bias=False,
                conv_cfg=conv_cfg,
                norm_cfg=norm_cfg,
                act_cfg=None))
        self.semantic_dwconv = nn.Sequential(
            DepthwiseSeparableConvModule(
                in_channels=self.out_channels,
                out_channels=self.out_channels,
                kernel_size=3,
                stride=1,
                padding=1,
                dw_norm_cfg=norm_cfg,
                dw_act_cfg=None,
                pw_norm_cfg=None,
                pw_act_cfg=None,
            ))
        self.conv = ConvModule(
            in_channels=self.out_channels,
            out_channels=self.out_channels,
            kernel_size=3,
            stride=1,
            padding=1,
            inplace=True,
            conv_cfg=conv_cfg,
            norm_cfg=norm_cfg,
            act_cfg=act_cfg,
        )

    def forward(self, x_d, x_s):
        detail_dwconv = self.detail_dwconv(x_d)
        detail_down = self.detail_down(x_d)
        semantic_conv = self.semantic_conv(x_s)
        semantic_dwconv = self.semantic_dwconv(x_s)
        semantic_conv = resize(
            input=semantic_conv,
            size=detail_dwconv.shape[2:],
            mode='bilinear',
            align_corners=self.align_corners)
        fuse_1 = detail_dwconv * torch.sigmoid(semantic_conv)
        fuse_2 = detail_down * torch.sigmoid(semantic_dwconv)
        fuse_2 = resize(
            input=fuse_2,
            size=fuse_1.shape[2:],
            mode='bilinear',
            align_corners=self.align_corners)
        output = self.conv(fuse_1 + fuse_2)
        return output


@MODELS.register_module()
class BiSeNetV2(BaseModule):
    """BiSeNetV2: Bilateral Network with Guided Aggregation for
    Real-time Semantic Segmentation.

    This backbone is the implementation of
    `BiSeNetV2 <https://arxiv.org/abs/2004.02147>`_.

    Args:
```

```
        in_channels (int): Number of channel of input image. Default: 3.
        detail_channels (Tuple[int], optional): Channels of each stage
            in Detail Branch. Default: (64, 64, 128).
        semantic_channels (Tuple[int], optional): Channels of each stage
            in Semantic Branch. Default: (16, 32, 64, 128).
            See Table 1 and Figure 3 of paper for more details.
        semantic_expansion_ratio (int, optional): The expansion factor
            expanding channel number of middle channels in Semantic Branch.
            Default: 6.
        bga_channels (int, optional): Number of middle channels in
            Bilateral Guided Aggregation Layer. Default: 128.
        out_indices (Tuple[int] | int, optional): Output from which stages.
            Default: (0, 1, 2, 3, 4).
        align_corners (bool, optional): The align_corners argument of
            resize operation in Bilateral Guided Aggregation Layer.
            Default: False.
        conv_cfg (dict | None): Config of conv layers.
            Default: None.
        norm_cfg (dict | None): Config of norm layers.
            Default: dict(type='BN').
        act_cfg (dict): Config of activation layers.
            Default: dict(type='ReLU').
        init_cfg (dict or list[dict], optional): Initialization config dict.
            Default: None.
    """

    def __init__(self,
                 in_channels=3,
                 detail_channels=(64, 64, 128),
                 semantic_channels=(16, 32, 64, 128),
                 semantic_expansion_ratio=6,
                 bga_channels=128,
                 out_indices=(0, 1, 2, 3, 4),
                 align_corners=False,
                 conv_cfg=None,
                 norm_cfg=dict(type='BN'),
                 act_cfg=dict(type='ReLU'),
                 init_cfg=None):
        if init_cfg is None:
            init_cfg = [
                dict(type='Kaiming', layer='Conv2d'),
                dict(
                    type='Constant', val=1, layer=['_BatchNorm', 'GroupNorm'])
            ]
        super().__init__(init_cfg=init_cfg)
        self.in_channels = in_channels
        self.out_indices = out_indices
        self.detail_channels = detail_channels
        self.semantic_channels = semantic_channels
        self.semantic_expansion_ratio = semantic_expansion_ratio
        self.bga_channels = bga_channels
        self.align_corners = align_corners
        self.conv_cfg = conv_cfg
        self.norm_cfg = norm_cfg
        self.act_cfg = act_cfg

        self.detail = DetailBranch(self.detail_channels, self.in_channels)
        self.semantic = SemanticBranch(self.semantic_channels,
                                       self.in_channels,
                                       self.semantic_expansion_ratio)
        self.bga = BGALayer(self.bga_channels, self.align_corners)

    def forward(self, x):
        #  stole refactoring code from Coin Cheung, thanks
        x_detail = self.detail(x)
        x_semantic_lst = self.semantic(x)
        x_head = self.bga(x_detail, x_semantic_lst[-1])
        outs = [x_head] + x_semantic_lst[:-1]
        outs = [outs[i] for i in self.out_indices]
```

```
        return tuple(outs)
```